Programs and Interpreters

The quadratic form as the interpreter of the Clifford algebra

Session 13b of three: 13a *Quadratic Forms and the Clifford Product*, 13c *Wedge, Dual, and Binding*

James Pustejovsky

Brandeis University
Department of Computer Science

CS 135: Computational Semantics

Fall 2026

Where This Picks Up

Session 13a declared a quadratic form, demanded $\mathbf{v}\mathbf{v} = Q(\mathbf{v})$, and got the Clifford product: two moves, swap and cancel, evaluate every word in the basis letters, and one word, $\mathbf{e}_2\mathbf{e}_1\mathbf{e}_2\mathbf{e}_2\mathbf{e}_1$, came out as $\mathbf{e}_2$, $-\mathbf{e}_2$ or 0 under three declarations. It ended with a slogan: the quadratic form is the specification; products of vectors are the programs.

Today we take the slogan literally. If products of vectors are programs, what is the language, what does it mean to run a program, and what job does the quadratic form do?

The claim of this session

The quadratic form is the **interpreter**. It supplies one rule, $\mathbf{v}\mathbf{v} = Q(\mathbf{v})$, which says what an instruction does when it meets itself. Every other behavior of the algebra – cycles, boosts, translations, mirrors, their inverses – is what that interpreter makes of the program text.

And since FGA declares a quadratic form for meaning, choosing the interpreter is choosing which *semantic* programs can run: polar pairs, cycles, coercions. Session 13c then builds two typed functions, the wedge and the dual, on top of this language, and Session 14 gives the full table of three-dimensional space.

Roadmap

I	The metaphor	programs, interpreters, and the puzzle of i
II	The interpreter is two rewrite rules	evaluation as sorting with a sign
III	One program, several interpreters	i^2 under four declarations; running on data
IV	Instructions and their undo	mirrors, inverses, null instructions; program equivalence
V	What the interpreter does not write	the wedge; three ways to supply an interpreter
VI	Interpreters for meaning	polar pairs, seasons, coercion, modification, polysemy
VII	Close	where the metaphor stops; exercises; reading

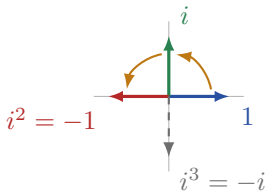
Every product on these slides is evaluated by hand, step by step, and checked by a short script.

Part I
The Metaphor

programs, interpreters, and the puzzle of i

A Puzzle: What Is i?

School presents i as a number, $\sqrt{-1}$, which no ruler can measure. Gauss's reading (1831) is different: i is an instruction, "turn a quarter turn."



Read that way, $i^2 = -1$ says: *run the quarter turn twice, and you face the other way.* A program is being run.

The question for today

Where does the -1 come from? Not from the instruction itself. Session 13a showed that the same symbol $\mathbf{e}_1\mathbf{e}_2$ squares to -1, $+1$, or 0 depending on what was declared. **The answer depends on who runs the program.**

The Vocabulary of Programs, Matched

programming notion	**in the Clifford algebra**	**example**
alphabet of instructions	orthogonal basis vectors	$\mathbf{e}_1$, $\mathbf{e}_2$, $\mathbf{e}_3$
program text	a product of vectors (or a sum of products)	$\mathbf{e}_2\mathbf{e}_1\mathbf{e}_2$
sequencing	the geometric product	$\mathbf{ba}$: first $\mathbf{a}$, then $\mathbf{b}$
interpreter	the quadratic form Q	$\mathbf{e}_1^2 = +1,\ \mathbf{e}_2^2 = -1$
evaluation	rewriting to a normal form	$\mathbf{e}_2\mathbf{e}_1\mathbf{e}_2 \to -\mathbf{e}_1\mathbf{e}_2\mathbf{e}_2 \to -Q(\mathbf{e}_2)\,\mathbf{e}_1$
value	a multivector in the blade basis	$-\mathbf{e}_1$
type of a value	grade	a vector: grade 1
undo	the inverse	$\mathbf{a}^{-1} = \mathbf{a}/Q(\mathbf{a})$
running a program on data	the sandwich	$R\,\mathbf{x}\,\widetilde{R}$

The next part makes "evaluation" and "value" exact. The rest of the deck uses each row.

Code Is Data

In most languages, programs and the data they act on are different kinds of thing. In the Clifford algebra they are not: a vector can be an instruction, and it can be the data an instruction acts on.

expression	**the program**	**the data**
$-\mathbf{a}\,\mathbf{x}\,\mathbf{a}^{-1}$	the vector $\mathbf{a}$, as a mirror	the vector $\mathbf{x}$
$R\,\mathbf{x}\,\widetilde{R}$	the rotor $R = \mathbf{ba}$, two mirrors	the vector $\mathbf{x}$
$R\,(\mathbf{x} \wedge \mathbf{y})\,\widetilde{R}$	the same rotor	a plane: a bivector

Lisp programmers call this property **homoiconicity**: programs are written in the language's own data structures.

A semantic parallel. In Montague semantics a function can take another function as its argument: a quantified noun phrase, of type $\langle\langle e, t\rangle, t\rangle$, takes the verb phrase as its argument rather than the reverse. The Clifford algebra goes further: *every* element can act, and every element can be acted on, with one product.

This is why one formula, the sandwich, will move vectors, planes, and volumes alike in Session 14.

Syntax and Interpretation, Once More

This course has separated syntax from interpretation since Session 1: a formal language says which expressions are well formed, and a model says what they mean. The same split appears here.

	model-theoretic semantics	**Clifford algebra**
expressions	sentences of a formal language	products of vectors
what fixes meaning	a model and its interpretation function	the declared Q
same expression, two choices	true in one model, false in another	$\mathbf{e}_{12}^2 = -1$ under one Q, $+1$ under another
what is never reinterpreted	the logical constants	associativity, distributivity, $\wedge$

One difference matters. A model interprets every non-logical word. The quadratic form interprets only one event: an instruction meeting itself. Everything else about a program is fixed by the language.

The last row anticipates Part V: the wedge is computed the same way under every Q.

Part II

The Interpreter Is Two Rewrite Rules

evaluation as sorting with a sign

The Alphabet and the Declaration

Fix an orthogonal basis $\mathbf{e}_1, \ldots, \mathbf{e}_n$, the alphabet. The declaration assigns each letter one of three values:

$$Q(\mathbf{e}_i) \in \{+1,\ -1,\ 0\}, \qquad \mathrm{Cl}(p,q,r): \quad p \text{ letters } +1, \quad q \text{ letters } -1, \quad r \text{ letters } 0$$

(Sylvester's theorem, Session 13a: for any quadratic form there is a basis in which it looks like this, and the three counts do not depend on the basis.)

declaration	**name**	**used for**
$\mathrm{Cl}(3,0,0)$	Euclidean space	ordinary rotations; FGA's first application
$\mathrm{Cl}(1,1,0)$, $\mathrm{Cl}(1,3,0)$	Lorentzian	boosts; special relativity; hyperbolic embeddings
$\mathrm{Cl}(0,2,0)$	–	the quaternions
$\mathrm{Cl}(3,0,1)$	projective	translations; the Geometric Algebra Transformer

The declaration is the whole interpreter. The rest of this part shows that two rules, both read off Q, evaluate every program.

Two Rewrite Rules

rule	rewrite	when
SWAP	$\mathbf{e}_j\mathbf{e}_i \longrightarrow -\mathbf{e}_i\mathbf{e}_j$	two different letters, $j > i$
COLLAPSE	$\mathbf{e}_i\mathbf{e}_i \longrightarrow Q(\mathbf{e}_i)$	a letter meets itself

COLLAPSE is the declaration itself. SWAP looks like a separate rule, but it also comes from Q, by polarization. Apply $\mathbf{v}\mathbf{v} = Q(\mathbf{v})$ to $\mathbf{v} = \mathbf{e}_i + \mathbf{e}_j$:

$$\mathbf{e}_i\mathbf{e}_i + \mathbf{e}_i\mathbf{e}_j + \mathbf{e}_j\mathbf{e}_i + \mathbf{e}_j\mathbf{e}_j = Q(\mathbf{e}_i + \mathbf{e}_j) = Q(\mathbf{e}_i) + Q(\mathbf{e}_j) \quad \Longrightarrow \quad \mathbf{e}_i\mathbf{e}_j + \mathbf{e}_j\mathbf{e}_i = 0$$

The middle step is orthogonality: $Q(\mathbf{e}_i + \mathbf{e}_j) = Q(\mathbf{e}_i) + Q(\mathbf{e}_j)$ when the cross term $2S(\mathbf{e}_i, \mathbf{e}_j)$ is 0.

Two jobs

SWAP reorders and costs a sign; it never produces a number. COLLAPSE is **the only rule that turns letters into a value**, and it is the only place the three declarations differ.

Evaluating a Program, Step by Step

Two traces in Euclidean space, $\mathrm{Cl}(3,0,0)$, where every $Q(\mathbf{e}_i) = +1$:

step	expression	rule
start	$\mathbf{e}_2\mathbf{e}_1\mathbf{e}_2$	
1	$-\mathbf{e}_1\mathbf{e}_2\mathbf{e}_2$	SWAP the first two
2	$-Q(\mathbf{e}_2)\,\mathbf{e}_1 = -\mathbf{e}_1$	COLLAPSE $\mathbf{e}_2\mathbf{e}_2$
start	$\mathbf{e}_1\mathbf{e}_2\mathbf{e}_3\mathbf{e}_1$	
1	$-\mathbf{e}_1\mathbf{e}_2\mathbf{e}_1\mathbf{e}_3$	SWAP $\mathbf{e}_3\mathbf{e}_1$
2	$+\mathbf{e}_1\mathbf{e}_1\mathbf{e}_2\mathbf{e}_3$	SWAP $\mathbf{e}_2\mathbf{e}_1$
3	$Q(\mathbf{e}_1)\,\mathbf{e}_2\mathbf{e}_3 = \mathbf{e}_{23}$	COLLAPSE $\mathbf{e}_1\mathbf{e}_1$

The first program is a reflection of $\mathbf{e}_1$ written as a product (Part IV). The second program has four instructions and evaluates to a plane: two letters survive, so its value has grade 2.

Evaluation Is Sorting With a Sign

SWAP is exactly the step of **bubble sort**: exchange two neighbors that are out of order. So evaluating a product of distinct letters means sorting it, and the sign counts the exchanges:

$$\text{sign} \;=\; (-1)^{\#\text{inversions}}, \qquad \text{inversion} = \text{a pair of letters in the wrong order}$$

program	inversions	count	value
$\mathbf{e}_3\mathbf{e}_1\mathbf{e}_2$	$(3,1)$, $(3,2)$	2	$+\,\mathbf{e}_{123}$
$\mathbf{e}_3\mathbf{e}_2\mathbf{e}_1$	$(3,2)$, $(3,1)$, $(2,1)$	3	$-\,\mathbf{e}_{123}$
$\mathbf{e}_2\mathbf{e}_3\mathbf{e}_1$	$(2,1)$, $(3,1)$	2	$+\,\mathbf{e}_{123}$

Why the order of rewriting does not matter

Any sequence of exchanges that sorts the word uses a number of exchanges with the same parity, because the parity of a permutation is fixed. So every evaluation order reaches the same value. In rewriting terms the system is **confluent**, and since each step shortens or sorts the word, it always **terminates**.

The Value of a Program

Putting both rules together, every product of basis vectors evaluates to a **normal form**:

$$\pm \Big(\prod_{\text{collapsed letters}} Q(\mathbf{e}_i) \Big) \, \mathbf{e}_{i_1} \mathbf{e}_{i_2} \cdots \mathbf{e}_{i_k}, \qquad i_1 < i_2 < \cdots < i_k$$

- **The survivors** are the letters that occur an odd number of times; the others collapse in pairs.
- **The grade** of the value is the number of survivors. There are 2^n possible sorted survivor sets, which is why $\mathrm{Cl}(n)$ has dimension 2^n.
- **The sign** comes from SWAP, and **the scalar** from COLLAPSE.

Worked: $\mathbf{e}_1\mathbf{e}_2\mathbf{e}_1\mathbf{e}_3\mathbf{e}_2$. Letter counts: $\mathbf{e}_1$ twice, $\mathbf{e}_2$ twice, $\mathbf{e}_3$ once. One survivor, so the value is $\pm\mathbf{e}_3$. Trace: $\mathbf{e}_1\mathbf{e}_2\mathbf{e}_1 \to -\mathbf{e}_1\mathbf{e}_1\mathbf{e}_2 \to -\mathbf{e}_2$; then $-\mathbf{e}_2\mathbf{e}_3\mathbf{e}_2 \to +\mathbf{e}_3\mathbf{e}_2\mathbf{e}_2 \to +\mathbf{e}_3$.

Grade as the type of a value

The grade of a value can be read off the program text without running it: count the letters with odd multiplicity. Its parity, even or odd, is preserved by every product. That is a type, in the sense of Session 12c: a property of the expression that constrains how it can be used.

Where the Interpreter Speaks

Run the same programs under three declarations. Only the second letter's declaration changes:

program	$\mathbf{e}_2^2 = +1$	$\mathbf{e}_2^2 = -1$	$\mathbf{e}_2^2 = 0$
$\mathbf{e}_2\mathbf{e}_1\mathbf{e}_2$	$-\mathbf{e}_1$	$+\mathbf{e}_1$	0
$\mathbf{e}_2\mathbf{e}_3\mathbf{e}_1\mathbf{e}_2$	$-\mathbf{e}_{13}$	$+\mathbf{e}_{13}$	0
$\mathbf{e}_1\mathbf{e}_2\mathbf{e}_3\mathbf{e}_1$	$\mathbf{e}_{23}$	$\mathbf{e}_{23}$	$\mathbf{e}_{23}$
$\mathbf{e}_3\mathbf{e}_2\mathbf{e}_1$	$-\mathbf{e}_{123}$	$-\mathbf{e}_{123}$	$-\mathbf{e}_{123}$

(All other letters declared $+1$.) The last two rows never collapse $\mathbf{e}_2$, so the declaration of $\mathbf{e}_2$ never enters: they come out the same under all three.

The interpreter speaks only at a collapse. A program whose letters are all distinct is pure sorting, and its value is the same under every declaration. A program that repeats a letter asks the interpreter what that letter does twice.

In-Class Exercise 1: Evaluate

Work in $\mathrm{Cl}(3,0,0)$ unless told otherwise. Show each rewrite.

1. $\mathbf{e}_3\mathbf{e}_2\mathbf{e}_3$.
2. $\mathbf{e}_2\mathbf{e}_1\mathbf{e}_3\mathbf{e}_2$.
3. Count the inversions in $\mathbf{e}_2\mathbf{e}_3\mathbf{e}_1$, and give its value without rewriting.
4. Without running it, give the grade of $\mathbf{e}_1\mathbf{e}_2\mathbf{e}_1\mathbf{e}_2\mathbf{e}_3\mathbf{e}_1\mathbf{e}_3$. Then evaluate it.
5. Evaluate $\mathbf{e}_2\mathbf{e}_1\mathbf{e}_2$ in $\mathrm{Cl}(1,0,1)$, where $\mathbf{e}_2^2 = 0$. What has the interpreter done to the program?

Five minutes. For item 4, the survivors are the letters with odd multiplicity.

In-Class Exercise 1: Answers

1. $\mathbf{e}_3\mathbf{e}_2\mathbf{e}_3 \to -\mathbf{e}_2\mathbf{e}_3\mathbf{e}_3 \to -Q(\mathbf{e}_3)\,\mathbf{e}_2 = -\mathbf{e}_2$.
2. Move the last $\mathbf{e}_2$ left past $\mathbf{e}_3$ and $\mathbf{e}_1$: two swaps, sign $+$. Then $\mathbf{e}_2\mathbf{e}_2\mathbf{e}_1\mathbf{e}_3 \to Q(\mathbf{e}_2)\,\mathbf{e}_1\mathbf{e}_3 = \mathbf{e}_{13}$.
3. Inversions $(2,1)$ and $(3,1)$: two, so $+\,\mathbf{e}_{123}$.
4. Counts: $\mathbf{e}_1$ three times, $\mathbf{e}_2$ twice, $\mathbf{e}_3$ twice. One survivor, $\mathbf{e}_1$: grade 1. Evaluating: $\mathbf{e}_1\mathbf{e}_2\mathbf{e}_1\mathbf{e}_2 = (\mathbf{e}_1\mathbf{e}_2)^2 = -1$; then $-\mathbf{e}_3\mathbf{e}_1\mathbf{e}_3 \to +\mathbf{e}_1\mathbf{e}_3\mathbf{e}_3 = \mathbf{e}_1$. Value $+\mathbf{e}_1$.
5. $\mathbf{e}_2\mathbf{e}_1\mathbf{e}_2 \to -\mathbf{e}_1\mathbf{e}_2\mathbf{e}_2 \to -Q(\mathbf{e}_2)\,\mathbf{e}_1 = 0$. A null letter collapses to nothing, so the whole program evaluates to 0: repeating a null instruction erases the result.

Part III
One Program, Several Interpreters

The Program $i = \mathbf{e}_1\mathbf{e}_2$

Run i twice. The evaluation is the same text under every declaration, up to the last step:

$$i^2 = \mathbf{e}_1\mathbf{e}_2\mathbf{e}_1\mathbf{e}_2 \xrightarrow{\text{SWAP}} -\,\mathbf{e}_1\mathbf{e}_1\mathbf{e}_2\mathbf{e}_2 \xrightarrow{\text{COLLAPSE}\ \times 2} -\,Q(\mathbf{e}_1)\,Q(\mathbf{e}_2)$$

interpreter	$Q(\mathbf{e}_1),\ Q(\mathbf{e}_2)$	i^2	**what i does**
Euclidean, $\mathrm{Cl}(2,0,0)$	$+1,\ +1$	-1	a quarter turn
Lorentzian, $\mathrm{Cl}(1,1,0)$	$+1,\ -1$	$+1$	a boost
degenerate, $\mathrm{Cl}(1,0,1)$	$+1,\ 0$	0	a translation
anti-Euclidean, $\mathrm{Cl}(0,2,0)$	$-1,\ -1$	-1	a quarter turn, again

One program text, three behaviors. The interpreter does not see the program as a whole; it sees two collapses and multiplies what each one returns. The sign in front came from SWAP, which every interpreter shares.

The last two rows agree because only the product $Q(\mathbf{e}_1)Q(\mathbf{e}_2)$ matters for i^2: two minus signs cancel.

Running i Again and Again

Powers of i, each computed with the two rules:

interpreter	i	i^2	i^3	i^4	**pattern**
Euclidean	$\mathbf{e}_{12}$	-1	$-\mathbf{e}_{12}$	1	a cycle of four
Lorentzian	$\mathbf{e}_{12}$	1	$\mathbf{e}_{12}$	1	alternates, period two
degenerate	$\mathbf{e}_{12}$	0	0	0	halts after one step

- **Euclidean**: four runs return to the start, as four quarter turns do. This is the loop a clock or a calendar needs.
- **Lorentzian**: $i^2 = 1$, so i undoes itself. The interesting behavior is not in the powers but in the exponential (next slide), which never comes back.
- **Degenerate**: once is something, twice is nothing. A null bivector is a program whose repetition contributes nothing new: change without accumulation.

The Degenerate Interpreter Carries a Rate

"Halts after one step" is what makes a null letter useful. Write ε for a letter declared $\varepsilon^2 = 0$, and run any polynomial on $a + \varepsilon$. Every term with ε^2 vanishes; what survives is the value and the derivative:

$$f(a + \varepsilon) \;=\; f(a) + f'(a)\,\varepsilon$$

program	**evaluation**	**reads as**
$(2+\varepsilon)^2$	$4 + 4\varepsilon + \varepsilon^2 = 4 + 4\varepsilon$	$x^2 = 4$, slope 4
$(2+\varepsilon)^3$	$8 + 12\varepsilon + 6\varepsilon^2 + \varepsilon^3 = 8 + 12\varepsilon$	$x^3 = 8$, slope 12
$(2+\varepsilon)^3 - 2(2+\varepsilon)$	$(8 + 12\varepsilon) - (4 + 2\varepsilon) = 4 + 10\varepsilon$	$x^3 - 2x = 4$, slope 10

And indeed $\frac{d}{dx}(x^3 - 2x) = 3x^2 - 2 = 10$ at $x = 2$. This is **forward-mode automatic differentiation**: PyTorch calls these objects dual tensors.

A null letter is a nonzero quantity whose square is zero: an infinitesimal, which records the direction of a change without contributing to its size. The interpreter that halts a repeated program keeps first-order information and nothing else.

The three one-letter interpreters of 13a divide the work: $+1$ splits a domain into two exclusive pieces (Part VI), -1 turns it in a cycle, 0 differentiates.

Running i Continuously: Exponentials

Running a program "a little at a time" is the exponential, $e^{B\theta} = \sum_k (B\theta)^k / k!$. Since every power of B is a number or a multiple of B, the series depends only on what the interpreter says B^2 is:

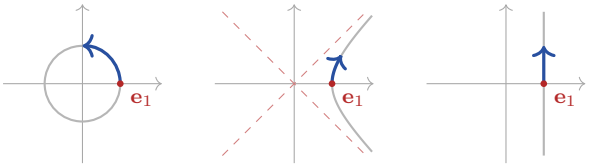
	$B^2 = -1$	$B^2 = +1$	$B^2 = 0$
$e^{B\theta}$	$\cos\theta + B\sin\theta$	$\cosh\theta + B\sinh\theta$	$1 + B\theta$
at $\theta = 0.7$	$0.765 + 0.644\,B$	$1.255 + 0.759\,B$	$1 + 0.7\,B$
as θ grows	periodic	grows without bound	grows linearly
kind	elliptic	hyperbolic	parabolic

Same program, same series, three different functions. The series is the same text in every column; the only difference is which value the interpreter supplies each time B meets itself.

The degenerate series stops after two terms, because every later term contains $B^2 = 0$: Part II's "a null letter erases the result," applied to an infinite program.

Running the Program on Data

A rotor $R = e^{-B\theta/2}$ acts on data by the sandwich $\mathbf{x} \mapsto R\,\mathbf{x}\,\widetilde{R}$. Follow $\mathbf{e}_1$ under each interpreter:



Euclidean: a circle Lorentzian: a hyperbola degenerate: a line

interpreter	$R\,\mathbf{e}_1\,\widetilde{R}$	**its declared length**
Euclidean, $\theta = 90°$	$\mathbf{e}_2$	1
Lorentzian, $\theta = \ln 2$	$1.25\,\mathbf{e}_1 + 0.75\,\mathbf{e}_2$	$1.25^2 - 0.75^2 = 1$
degenerate, $\theta = 1$	$\mathbf{e}_1 + \mathbf{e}_2$	$1 + 0 = 1$

Each program slides its data along the set $Q = 1$ of its own interpreter. That set is a circle, a hyperbola, or a line, and the program never leaves it.

Other Programs, the Same Lesson

Two more programs, each evaluated under several interpreters:

program	$(+,+)$	$(+,-)$	$(+,0)$	$(-,-)$
$(\mathbf{e}_1+\mathbf{e}_2)^2 = Q(\mathbf{e}_1)+Q(\mathbf{e}_2)$	2	0	1	-2

program	$(+,+,+)$	$(+,+,-)$	$(+,-,-)$	$(+,+,0)$
$\mathbf{e}_{123}^2 = -\,Q(\mathbf{e}_1)Q(\mathbf{e}_2)Q(\mathbf{e}_3)$	-1	$+1$	-1	0

First row: under the Lorentzian interpreter, $\mathbf{e}_1+\mathbf{e}_2$ squares to 0. It is a nonzero vector that the interpreter treats as having no length: a **null** instruction. Part IV shows that it has no undo.

Second row: $\mathbf{e}_{123}^2$ needs three swaps, so the sign is $-$, and three collapses. In Euclidean 3D the pseudoscalar squares to -1, which is why it can play the part of i for duality (Session 13c).

The Complex Numbers and the Quaternions, Recompiled

The complex numbers. Two ways to get them as programs:

- one letter, declared $\mathbf{e}^2 = -1$: elements $a + b\,\mathbf{e}$, with $(a + b\,\mathbf{e})(c + d\,\mathbf{e}) = (ac - bd) + (ad + bc)\,\mathbf{e}$;
- the even programs of the Euclidean plane: $a + b\,\mathbf{e}_{12}$, with $\mathbf{e}_{12}^2 = -1$.

The quaternions. In $\mathrm{Cl}(0,2,0)$, let $i = \mathbf{e}_1$, $j = \mathbf{e}_2$, $k = \mathbf{e}_1\mathbf{e}_2$. Hamilton's rule $ijk = -1$ is an evaluation:

$$ijk = \mathbf{e}_1\,\mathbf{e}_2\,\mathbf{e}_1\mathbf{e}_2 \;\rightarrow\; -\,\mathbf{e}_1\mathbf{e}_1\mathbf{e}_2\mathbf{e}_2 \;\rightarrow\; -\,Q(\mathbf{e}_1)\,Q(\mathbf{e}_2) = -\,(-1)(-1) = -1$$

and $ij = \mathbf{e}_1\mathbf{e}_2 = k$, while $ji = \mathbf{e}_2\mathbf{e}_1 = -\mathbf{e}_1\mathbf{e}_2 = -k$: the noncommutativity Hamilton discovered in 1843 is SWAP.

What was "discovered"

Neither number system has to be postulated. Each is what a particular interpreter makes of a small alphabet: the complex numbers from one letter declared -1, the quaternions from two. Session 13a tabulated the family; Session 14 finds the quaternions again among the three planes of ordinary space.

$i^2 = -1$ Is a Fact About the Interpreter

what belongs to the program	**what belongs to the interpreter**
the letters and their order	what each letter returns when it meets itself
the sign from reordering	every scalar that appears in a value
which letters survive: the grade	whether a repeated program cycles, grows, or stops
the wedge, $\mathbf{u} \wedge \mathbf{v}$ (Part V)	the symmetric half, $S(\mathbf{u}, \mathbf{v})$

So $i^2 = -1$ is not a fact about i. It is what the Euclidean interpreter does with the program $\mathbf{e}_1\mathbf{e}_2$. Under a Lorentzian interpreter the same text gives $+1$, and under a degenerate one, 0.

This is the point to carry into Part VI. When FGA declares a quadratic form for meaning, it is not choosing coordinates; it is choosing an interpreter, and with it which programs cycle, which grow, and which can be undone.

Part IV
Instructions and Their Undo

A Vector Is an Instruction: Reflect

A vector $\mathbf{a}$ with $Q(\mathbf{a}) \neq 0$ is a program that reflects its data in the line (or hyperplane) orthogonal to $\mathbf{a}$:

$$\mathbf{x} \longmapsto -\mathbf{a}\,\mathbf{x}\,\mathbf{a}^{-1}, \qquad \mathbf{a}^{-1} = \frac{\mathbf{a}}{Q(\mathbf{a})}$$

Worked, Euclidean plane: $\mathbf{a} = \mathbf{e}_1 + \mathbf{e}_2$, so $Q(\mathbf{a}) = 2$ and $\mathbf{a}^{-1} = \frac{1}{2}(\mathbf{e}_1 + \mathbf{e}_2)$. Data $\mathbf{x} = \mathbf{e}_1$:

step	expression
$\mathbf{a}\,\mathbf{e}_1$	$(\mathbf{e}_1 + \mathbf{e}_2)\,\mathbf{e}_1 = \mathbf{e}_1\mathbf{e}_1 + \mathbf{e}_2\mathbf{e}_1 = 1 - \mathbf{e}_{12}$
$(1 - \mathbf{e}_{12})(\mathbf{e}_1 + \mathbf{e}_2)$	$\mathbf{e}_1 + \mathbf{e}_2 - \mathbf{e}_{12}\mathbf{e}_1 - \mathbf{e}_{12}\mathbf{e}_2 = \mathbf{e}_1 + \mathbf{e}_2 + \mathbf{e}_2 - \mathbf{e}_1 = 2\,\mathbf{e}_2$
times $-\frac{1}{2}$	$-\,\mathbf{e}_2$

$\mathbf{e}_1 = (1, 0)$ is reflected in the line $y = -x$, landing at $(0, -1)$. The instruction is the mirror's normal; the interpreter's value $Q(\mathbf{a}) = 2$ is what makes it undoable.

In the second row, $\mathbf{e}_{12}\mathbf{e}_1 = \mathbf{e}_1\mathbf{e}_2\mathbf{e}_1 = -\mathbf{e}_1\mathbf{e}_1\mathbf{e}_2 = -\mathbf{e}_2$ and $\mathbf{e}_{12}\mathbf{e}_2 = \mathbf{e}_1\mathbf{e}_2\mathbf{e}_2 = \mathbf{e}_1$.

Undo Is Running the Program Backwards

For a program $P = \mathbf{a}_1\mathbf{a}_2\cdots\mathbf{a}_k$, its **reverse** $\widetilde{P} = \mathbf{a}_k\cdots\mathbf{a}_2\mathbf{a}_1$ writes the instructions in the opposite order. Then the middle collapses, pair by pair:

$$P\,\widetilde{P} = \mathbf{a}_1\cdots\mathbf{a}_{k-1}\,(\mathbf{a}_k\mathbf{a}_k)\,\mathbf{a}_{k-1}\cdots\mathbf{a}_1 = Q(\mathbf{a}_k)\,\mathbf{a}_1\cdots(\mathbf{a}_{k-1}\mathbf{a}_{k-1})\cdots\mathbf{a}_1 = \cdots = \prod_i Q(\mathbf{a}_i)$$

The undo rule

$P^{-1} = \widetilde{P}\,/\prod_i Q(\mathbf{a}_i)$: run the instructions backwards, then divide by what the interpreter returned at each collapse. The undo exists exactly when no instruction is null.

P	$\widetilde{P}$	$P\,\widetilde{P}$	P^{-1}
ba, two unit mirrors	**ab**	1	**ab**
$(\mathbf{e}_1+\mathbf{e}_2)(2\mathbf{e}_1) = 2-2\mathbf{e}_{12}$	$2+2\mathbf{e}_{12}$	$2\cdot 4 = 8$	$\frac{1}{8}(2+2\mathbf{e}_{12})$

In the second row, $8 = Q(\mathbf{e}_1+\mathbf{e}_2)\,Q(2\mathbf{e}_1)$. For unit mirrors the undo is the reverse itself, $R^{-1} = \widetilde{R}$, so $R\,\mathbf{x}\,\widetilde{R}$ is undone by $\widetilde{R}\,\mathbf{x}\,R$.

Instructions With No Undo

If some instruction has $Q(\mathbf{a}) = 0$, the product of collapses is 0, and there is nothing to divide by.

A null instruction. Lorentzian plane, $\mathbf{a} = \mathbf{e}_1 + \mathbf{e}_2$:

$$\mathbf{a}^2 = Q(\mathbf{e}_1) + Q(\mathbf{e}_2) = 1 - 1 = 0, \qquad \mathbf{a}\,\mathbf{e}_1\,\mathbf{a} = 2\,\mathbf{e}_1 + 2\,\mathbf{e}_2 = 2\,\mathbf{a}$$

Wrapped around any data, $\mathbf{a}$ returns a multiple of itself: whatever went in, the original cannot be recovered.

The empty interpreter. Declare every $Q(\mathbf{e}_i) = 0$. Then COLLAPSE always returns 0:

$$\mathbf{e}_1\mathbf{e}_1 = 0, \qquad (\mathbf{e}_{12})^2 = 0, \qquad (\mathbf{e}_1 + \mathbf{e}_2)^2 = 0, \qquad \text{but} \qquad \mathbf{e}_1\mathbf{e}_2 = \mathbf{e}_{12} \neq 0$$

This is Grassmann's exterior algebra. Programs with distinct letters still have values – oriented lines, planes, volumes – but no instruction can be undone and nothing turns.

Programs that can describe but cannot act. The wedge alone names oriented subspaces; it takes a nonzero interpreter for those subspaces to become mirrors and rotors.

Composing Instructions: Two Mirrors Make a Rotor

Sequencing two reflections is one program, the product. Euclidean plane, mirrors $\mathbf{a} = \mathbf{e}_1$ and $\mathbf{b} = \frac{1}{\sqrt{2}}(\mathbf{e}_1 + \mathbf{e}_2)$, data $3\mathbf{e}_1 + \mathbf{e}_2$:

step	**instruction**	**data**
start	–	$3\mathbf{e}_1 + \mathbf{e}_2$
reflect	$\mathbf{a} = \mathbf{e}_1$	$-3\mathbf{e}_1 + \mathbf{e}_2$
reflect again	$\mathbf{b} = \frac{1}{\sqrt{2}}(\mathbf{e}_1 + \mathbf{e}_2)$	$-\mathbf{e}_1 + 3\mathbf{e}_2$
in one step	$R = \mathbf{ba} = \frac{1}{\sqrt{2}}(1 - \mathbf{e}_{12})$	$R\,(3\mathbf{e}_1 + \mathbf{e}_2)\,\widetilde{R} = -\mathbf{e}_1 + 3\mathbf{e}_2$

Two reflections compose to a rotation by twice the angle between the mirrors: $2 \times 45° = 90°$. A rotor is a compiled two-instruction program.

Order matters: $\mathbf{ab} = \frac{1}{\sqrt{2}}(1 + \mathbf{e}_{12})$ rotates the other way. Sequencing is not commutative, because SWAP costs a sign.

When Are Two Programs the Same?

Programming languages distinguish two notions of equality, and so does this one:

equality	**test**	**example**
same value	the normal forms agree	$\mathbf{e}_3\mathbf{e}_1\mathbf{e}_2$ and $\mathbf{e}_2\mathbf{e}_3\mathbf{e}_1$: both $+\mathbf{e}_{123}$
same behavior	the sandwiches agree on all data	R and $-R$

Same value implies same behavior, but not the reverse: $(-R)\,\mathbf{x}\,(-\widetilde{R}) = R\,\mathbf{x}\,\widetilde{R}$, since the two signs cancel. Every rotation is run by exactly two rotors, R and $-R$.

Worked. The quarter-turn rotor $R = \frac{1}{\sqrt{2}}(1 - \mathbf{e}_{12})$ has $R^2 = -\mathbf{e}_{12}$ and $R^4 = -1$. Four quarter turns bring every vector back, $R^4\,\mathbf{x}\,\widetilde{R^4} = \mathbf{x}$, yet the program R^4 has value -1, not 1. Eight runs give value 1.

Two programs with the same behavior and different values is a distinction this course knows from Frege: two expressions can agree in what they pick out and differ in how they pick it out. Here the difference is invisible in the action on vectors, and it matters again for spinors and for composing rotations.

What the Interpreter Decided

declared	**consequence in the algebra**	**example**
$Q(\mathbf{a}) \neq 0$	$\mathbf{a}$ is invertible, so it can be a mirror	any Euclidean vector
$Q(\mathbf{a}) = 0$, $\mathbf{a} \neq \mathbf{0}$	no inverse; change without size	dual numbers; Lorentzian $\mathbf{e}_1 + \mathbf{e}_2$
one letter, $\mathbf{e}^2 = +1$	projectors $\frac{1}{2}(1 \pm \mathbf{e})$	a polar pair, alive/dead (Part VI)
one letter, $\mathbf{e}^2 = -1$	cycles	the complex numbers
one letter, $\mathbf{e}^2 = 0$	rates	dual numbers; autodiff
a bivector, $B^2 = -1$	elliptic rotors: rotations	$\mathrm{Cl}(2,0,0)$, $\mathrm{Cl}(0,2,0)$
a bivector, $B^2 = +1$	hyperbolic rotors: boosts	$\mathrm{Cl}(1,1,0)$
a bivector, $B^2 = 0$	translations	$\mathrm{Cl}(1,0,1)$, $\mathrm{Cl}(3,0,1)$

Every row is a consequence of one declaration. Choosing a signature is choosing which of these behaviors a space will have: for physics, for graphics, or, in Part VI, for meaning. None of it needed an antisymmetric form to be declared; the orientation in every rotor came from the program text, which is Part V.

Part V
What the Interpreter Does Not Write

Two Halves of One Product

The product of two vectors splits into a symmetric and an antisymmetric half (Sessions 11 and 13a):

$$\mathbf{uv} = S(\mathbf{u},\mathbf{v}) + \mathbf{u}\wedge\mathbf{v}, \qquad S = \tfrac{1}{2}(\mathbf{uv}+\mathbf{vu}), \quad \mathbf{u}\wedge\mathbf{v} = \tfrac{1}{2}(\mathbf{uv}-\mathbf{vu})$$

Worked: $\mathbf{u} = \mathbf{e}_1 + 2\mathbf{e}_2$, $\mathbf{v} = 3\mathbf{e}_1 + \mathbf{e}_2$, under four interpreters:

interpreter	$\mathbf{uv}$	$S(\mathbf{u},\mathbf{v})$	$\mathbf{u}\wedge\mathbf{v}$
Euclidean $(+,+)$	$5 - 5\,\mathbf{e}_{12}$	$3 + 2 = 5$	$-5\,\mathbf{e}_{12}$
Lorentzian $(+,-)$	$1 - 5\,\mathbf{e}_{12}$	$3 - 2 = 1$	$-5\,\mathbf{e}_{12}$
degenerate $(+,0)$	$3 - 5\,\mathbf{e}_{12}$	$3 + 0 = 3$	$-5\,\mathbf{e}_{12}$
empty $(0,0)$	$-5\,\mathbf{e}_{12}$	0	$-5\,\mathbf{e}_{12}$

The symmetric half changes with every interpreter; the wedge never does. It is $(1\cdot 1 - 2\cdot 3)\,\mathbf{e}_{12} = -5\,\mathbf{e}_{12}$ in every row.

Orientation Lives in the Program Text

Why the wedge never changes: expand $\mathbf{uv}$ letter by letter. The cross terms $\mathbf{e}_1\mathbf{e}_2$ and $\mathbf{e}_2\mathbf{e}_1$ are distinct letters, so they meet only SWAP, never COLLAPSE:

$$\mathbf{u} \wedge \mathbf{v} = (u_1 v_2 - u_2 v_1)\, \mathbf{e}_{12} \qquad \text{uses SWAP only}$$

$$S(\mathbf{u}, \mathbf{v}) = u_1 v_1\, Q(\mathbf{e}_1) + u_2 v_2\, Q(\mathbf{e}_2) \qquad \text{uses COLLAPSE only}$$

half	**answers**	**written by**
$S(\mathbf{u}, \mathbf{v})$	symmetric compatibility	the interpreter
$\mathbf{u} \wedge \mathbf{v}$	directional contribution, and which plane	the program text: the order of instructions

Session 11 split an attention head's matrix into $G = S + A$: the symmetric part is the compatibility of the pair, the antisymmetric part its directional contribution. The Clifford algebra declares the first and gets the second for free, from the order in which the instructions were written.

Three Ways to Supply an Interpreter

Every system this course has studied combines vectors with something that plays the interpreter's part:

	Clifford algebra	**HRR**	**attention head**
the interpreter is	declared: Q	fixed: arithmetic mod n	learned: $G = W_Q^\top W_K$
SWAP costs	a sign	nothing: $\mathbf{u} \circledast \mathbf{v} = \mathbf{v} \circledast \mathbf{u}$	– (no product to swap)
orientation	from SWAP: the wedge	none: $\mathbf{u} \circledast \mathbf{v} - \mathbf{v} \circledast \mathbf{u} = \mathbf{0}$	the learned part A
programs compose	yes, closed under product	yes, at fixed width	one step: a score
undo	$\widetilde{P}/\prod Q$	exact for unitary keys	none in general

Checked: for $\mathbf{u} = (1, 2, 0, 1)$, $\mathbf{v} = (2, 0, 1, 1)$, both orders of convolution give $(4, 5, 2, 5)$. An algebra whose swap is free has no antisymmetric half. The columns differ on one question: should the interpreter be fixed by arithmetic, learned, or declared?

Part VI
Interpreters for Meaning

which semantic programs a declaration lets run

FGA Declares an Interpreter for Meaning

Functional Geometric Algebra represents meanings in a Clifford algebra $\mathrm{Cl}(V, Q)$, with V divided into orthogonal subspaces for entities, argument slots, predicate content, roles and states. The position paper treats the signature as a design parameter and proposes:

axis squares to	**proposed use**	**programs it allows**
$+1$	spatial, extensional, similarity-preserving	projectors; rotations, in pairs
-1	intensional, modal, contrastive	cycles; boosts, with a $+1$ axis
0	not used for meaning	translations

Its first application uses the Euclidean signature, with unit entity vectors, so that the symmetric half of a product of two entities is their cosine.

In this deck's terms: the theorist picks the interpreter, and the interpreter decides which semantic programs can run. The next five slides take one program each, and ask which declaration it needs.

A Polar Pair: A Program That Is Its Own Repetition

alive and dead partition a domain: nothing is both, and (for the things they apply to) everything is one or the other. On an axis $\mathbf{e}$ declared $+1$:

$$\mathbf{p}_{\text{alive}} = \tfrac{1}{2}(1+\mathbf{e}), \qquad \mathbf{p}_{\text{dead}} = \tfrac{1}{2}(1-\mathbf{e})$$

property	**evaluation**	$\mathbf{e}^2=+1$	$\mathbf{e}^2=-1$
idempotent	$\mathbf{p}_{\text{alive}}^2 = \tfrac{1}{4}(1+2\mathbf{e}+\mathbf{e}^2)$	$\mathbf{p}_{\text{alive}}$	$\tfrac{1}{2}\mathbf{e}$
exclusive	$\mathbf{p}_{\text{alive}}\mathbf{p}_{\text{dead}} = \tfrac{1}{4}(1-\mathbf{e}^2)$	0	$\tfrac{1}{2}$
exhaustive	$\mathbf{p}_{\text{alive}} + \mathbf{p}_{\text{dead}}$	1	1

As a program: an idempotent is a filter, and running a filter twice is running it once. That is Session 11b's intersective predication, student $\odot$ student $=$ student.

It needs the $+1$ interpreter. Under $\mathbf{e}^2=-1$ the same program text is neither idempotent nor exclusive: the interpreter turned a filter into something else.

Seasons: A Program That Comes Back

Cyclic expressions – *next season*, *the season after next*, *a year from now* – need a program that returns to its start. In a Euclidean plane, let $R = \frac{1}{\sqrt{2}}(1 - \mathbf{e}_{12})$, a quarter turn, and file SPRING under $\mathbf{e}_1$:

expression	**program**	**value**	**lands on**
next season	R	$\frac{1}{\sqrt{2}}(1 - \mathbf{e}_{12})$	$\mathbf{e}_2$: SUMMER
the season after next	R^2	$-\mathbf{e}_{12}$	$-\mathbf{e}_1$: AUTUMN
the season before	R^3	$-\frac{1}{\sqrt{2}}(1 + \mathbf{e}_{12})$	$-\mathbf{e}_2$: WINTER
the season before	$R^{-1} = \widetilde{R}$	$+\frac{1}{\sqrt{2}}(1 + \mathbf{e}_{12})$	$-\mathbf{e}_2$: WINTER
a year from now	R^4	-1	$\mathbf{e}_1$: SPRING

The season before has two programs, three steps forward or one step back: opposite values, same behavior, since $R^3 = -\widetilde{R}$. *A year from now* has value -1 and the behavior of doing nothing. Part IV's distinction, in temporal expressions.

The loop needs a bivector that squares to -1: a Euclidean (or anti-Euclidean) plane. A Lorentzian plane would send SPRING along a hyperbola, never to return.

Coercion: A Program That Changes the Type

Mary began the book. Begin selects an event; book denotes an object. Generative Lexicon resolves the clash through the noun's qualia, here the telic role, reading. FGA writes this as a rotor applied to the argument before the predicate reads it.

A toy: a formal direction $\mathbf{f}$ and a telic direction $\mathbf{t}$, orthonormal, both declared $+1$. Let book $= \mathbf{f}$, and let begin look for $\mathbf{t}$.

the program, as two mirrors	$R = \frac{1}{\sqrt{2}}(\mathbf{f} + \mathbf{t})\,\mathbf{f} = \frac{1}{\sqrt{2}}(1 - \mathbf{ft})$
without the program	$\text{begin} \cdot \text{book} = \mathbf{t} \cdot \mathbf{f} = 0$: a type clash
with the program	$R\,\mathbf{f}\,\widetilde{R} = \mathbf{t}$, then $\text{begin} \cdot \mathbf{t} = 1$

Coercion is a program run on the argument. The first mirror is the formal direction itself; the second lies halfway between formal and telic. Their product turns the object reading into the event reading. This is the 90° rotation of Part IV with the axes renamed, and the quarter turn of the seasons slide.

Modification: A Program That Never Comes Back

Declare a form axis $\mathbf{e}_1^2 = +1$ and a telic axis $\mathbf{e}_2^2 = -1$. Now $\mathbf{e}_{12}^2 = +1$, and the rotor is a boost. With $\varphi = \ln 2$:

$$R = e^{-\mathbf{e}_{12}\varphi/2} = 1.0607 - 0.3536\,\mathbf{e}_{12}, \qquad R\,\mathbf{e}_1\,\widetilde{R} = 1.25\,\mathbf{e}_1 + 0.75\,\mathbf{e}_2$$

Its declared length is $1.25^2 - 0.75^2 = 1$: the boost preserves the interpreter's length, not the one on the page (where it is 1.46).

In FGA, the position paper uses this boost for good knife: boosting the form factor and rebuilding the form–purpose binding gives

$$(R\,\mathbf{e}_1\,\widetilde{R}) \wedge \mathbf{e}_2 = 1.25\,(\mathbf{e}_1 \wedge \mathbf{e}_2)$$

the same plane, bound 1.25 times as strongly. Unlike the seasons, repeating the program never returns: each boost moves further along the hyperbola.

The paper offers the contrast between elliptic and hyperbolic programs as an encoding hypothesis for extensional versus intensional modification. It is a proposal about which interpreter suits which phenomenon, to be tested.

Regular Polysemy: One Program, Many Words

Some sense alternations are regular: chicken, lamb, turkey as animals and as food. The position paper proposes one rotor per alternation, applied to every word that has it.

A toy: axes ANIMAL, FOOD, BIRD; the program turns 90° in the ANIMAL–FOOD plane.

	as an animal	**as food**
chicken	$(0.8,\ 0,\ 0.6)$	$(0,\ 0.8,\ 0.6)$
lamb	$(1,\ 0,\ 0)$	$(0,\ 1,\ 0)$
$\cos(\text{chicken}, \text{lamb})$	0.80	0.80

A shared program preserves every similarity among the words it runs on, because a rotor preserves the interpreter's form. That makes the proposal testable: are the food senses of lamb and chicken exactly as similar as their animal senses?

If not, each word needs its own program, or the alternation is not a rotor at all. That is course project E4.

What an Interpreter Rules Out

Every rotor preserves Q. So the interpreter limits which programs exist, not only how they behave:

if the declaration is	**then no program can**
$\mathbf{f}$ positive, $\mathbf{t}$ negative	turn $\mathbf{f}$ entirely into $\mathbf{t}$: a boost adds a telic part, $\cosh\varphi\,\mathbf{f} + \sinh\varphi\,\mathbf{t}$, but never removes the formal one
an axis $\mathbf{e}$ with $\mathbf{e}^2 = -1$	build an exclusive, exhaustive pair from it
a Lorentzian plane	make a cycle in that plane
a null direction	be undone, if it uses that direction as a mirror

A linguistic reading of the first row. If the theorist declares the telic dimension negative, coercion by that dimension can only *add* an event reading to an object; it cannot replace the object reading. Whether *began the book* keeps the book as an object, or loses it, is then an empirical question about the declaration.

An interpreter is a set of predictions: it says which semantic operations are available and which are impossible.

Choosing an Interpreter Is a Hypothesis About Meaning

program	**phenomenon**	**interpreter it needs**	**status**
idempotent pair	polar adjectives, alive/dead	a $+1$ axis	proposed
quarter-turn rotor	cyclic time expressions	a plane with $B^2 = -1$	toy
two-mirror rotor	coercion, *began the book*	both axes $+1$	proposed
boost	intensional modification, good knife	a $+1$ and a -1 axis	hypothesis
shared rotor	regular polysemy	a Euclidean plane	testable: E4

Read across, each row is a claim: *this phenomenon behaves like this program, so the meaning space must be declared this way*. Read down, different phenomena ask for different interpreters on different subspaces; FGA's answer is a mixed signature, declared axis by axis.

Part VII
Close

Where the Metaphor Stops

The metaphor is exact about some things and loose about others:

exact	**loose**
evaluation is rewriting with two rules, confluent and terminating	there are no conditionals, loops or variables: every program is straight-line
Q fixes the result of every collapse, and nothing else	"running" covers two different things: evaluating a product, and applying a sandwich
undo is reversal divided by $\prod Q$	"interpreter" here means a rewriting semantics, not a machine with state

The language is not meant to express everything a programming language can. What it does express, composition, orientation, reversibility, cycles, it expresses with one product and one declaration.

For real programming languages, Abelson and Sussman's version of the idea: an evaluator, which determines the meaning of expressions, is just another program. Here it is a quadratic form. 13c adds two typed functions, the wedge and the dual, and reads the cross product as a macro built from them.

Summary

- **Products of vectors are programs**; the geometric product is sequencing; a vector can be both an instruction and the data it acts on.
- **The interpreter is two rewrite rules.** SWAP reorders at the cost of a sign; COLLAPSE returns $Q(\mathbf{e}_i)$ when a letter meets itself. Both come from the declared Q. Evaluation is sorting with a sign, and the grade of the value is the number of letters with odd multiplicity.
- **One program, several interpreters**: $i = \mathbf{e}_1\mathbf{e}_2$ squares to -1, $+1$ or 0, and its exponential is periodic, unbounded or linear. $i^2 = -1$ is a fact about the Euclidean interpreter.
- **Undo is reversal**: $P^{-1} = \widetilde{P}/\prod Q(\mathbf{a}_i)$, which exists exactly when no instruction is null. Programs can share a behavior and differ in value: R and $-R$; $R^4 = -1$.
- **The interpreter does not write orientation**: the wedge comes from SWAP alone and is the same under every Q. HRR, whose swap is free, has no wedge.
- **For meaning**, choosing an interpreter is choosing which semantic programs can run: filters for polar pairs, cycles for seasons, rotors for coercion and polysemy, boosts for modification. Each choice is a hypothesis.

In-Class Exercise 2: Interpreters and Undo

1. Compute i^3 for $i = \mathbf{e}_1\mathbf{e}_2$ under the Euclidean, Lorentzian and degenerate interpreters.
2. Euclidean plane: find the inverse of $P = \mathbf{e}_1(\mathbf{e}_1 + \mathbf{e}_2)$ by reversing it. Check that $P\,P^{-1} = 1$.
3. Lorentzian plane: is $\mathbf{e}_1 + \mathbf{e}_2$ invertible? Is $\mathbf{e}_1 + 2\mathbf{e}_2$? Give the inverse where it exists.
4. For $\mathbf{u} = 2\mathbf{e}_1 + \mathbf{e}_2$ and $\mathbf{v} = \mathbf{e}_1 + \mathbf{e}_2$, compute $S(\mathbf{u}, \mathbf{v})$ and $\mathbf{u} \wedge \mathbf{v}$ under the Euclidean and the Lorentzian interpreter. Which one changed?
5. Use a null letter to find the derivative of $f(x) = x^4 - 3x$ at $x = 1$, and check it by hand.

Seven minutes in pairs.

In-Class Exercise 2: Answers

1. $i^3 = i^2\, i = -Q(\mathbf{e}_1)Q(\mathbf{e}_2)\,\mathbf{e}_{12}$: Euclidean $-\mathbf{e}_{12}$; Lorentzian $+\mathbf{e}_{12}$; degenerate 0.
2. $P = \mathbf{e}_1\mathbf{e}_1 + \mathbf{e}_1\mathbf{e}_2 = 1 + \mathbf{e}_{12}$. Reverse: $\widetilde{P} = (\mathbf{e}_1 + \mathbf{e}_2)\mathbf{e}_1 = 1 - \mathbf{e}_{12}$. The collapses return $Q(\mathbf{e}_1)\,Q(\mathbf{e}_1 + \mathbf{e}_2) = 1 \cdot 2 = 2$, so $P^{-1} = \frac{1}{2}(1 - \mathbf{e}_{12})$. Check: $(1 + \mathbf{e}_{12})(1 - \mathbf{e}_{12}) = 1 - \mathbf{e}_{12}^2 = 2$, and $2/2 = 1$.
3. $(\mathbf{e}_1 + \mathbf{e}_2)^2 = 1 - 1 = 0$: null, no inverse. $(\mathbf{e}_1 + 2\mathbf{e}_2)^2 = 1 - 4 = -3$, so $(\mathbf{e}_1 + 2\mathbf{e}_2)^{-1} = -\frac{1}{3}(\mathbf{e}_1 + 2\mathbf{e}_2)$.
4. $\mathbf{u} \wedge \mathbf{v} = (2 \cdot 1 - 1 \cdot 1)\,\mathbf{e}_{12} = \mathbf{e}_{12}$ under both. S is $2 + 1 = 3$ (Euclidean) and $2 - 1 = 1$ (Lorentzian). Only the symmetric half changed: the interpreter wrote it.
5. $(1 + \varepsilon)^4 - 3(1 + \varepsilon) = (1 + 4\varepsilon) - (3 + 3\varepsilon) = -2 + \varepsilon$: value -2, derivative 1. By hand, $4x^3 - 3 = 1$ at $x = 1$.

In-Class Exercise 3: Interpreters for Meaning

1. Show that $\mathbf{p}_{\text{dead}} = \frac{1}{2}(1 - \mathbf{e})$ is idempotent when $\mathbf{e}^2 = +1$. What is $\mathbf{p}_{\text{dead}}^2$ when $\mathbf{e}^2 = -1$?
2. With the seasons rotor $R = \frac{1}{\sqrt{2}}(1 - \mathbf{e}_{12})$, compute $\widetilde{R}\,\mathbf{e}_1\,R$. Which season is it, and which phrase does it translate?
3. In the coercion toy, suppose the telic axis is declared $\mathbf{t}^2 = -1$. Explain, using what a rotor preserves, why no rotor can send book $= \mathbf{f}$ exactly to $\mathbf{t}$.
4. Design a test for the shared-rotor account of chicken and lamb. What result would count against it?
5. $\star$ You are designing a semantic space that must hold a polar pair (alive/dead) and a cyclic domain (the seasons). Which sign would you declare for each axis, and why? What goes wrong if the two are swapped?

In-Class Exercise 3: Answers

1. $\frac{1}{4}(1-\mathbf{e})^2 = \frac{1}{4}(1-2\mathbf{e}+\mathbf{e}^2)$. With $\mathbf{e}^2=+1$: $\frac{1}{4}(2-2\mathbf{e}) = \frac{1}{2}(1-\mathbf{e})$, idempotent. With $\mathbf{e}^2=-1$: $-\frac{1}{2}\mathbf{e}$, not a filter.
2. $\widetilde{R} = \frac{1}{\sqrt{2}}(1+\mathbf{e}_{12})$, and $\widetilde{R}\,\mathbf{e}_1\,R = -\mathbf{e}_2$: WINTER, *the season before*. Running the program's reverse undoes one step.
3. A rotor preserves Q. $Q(\mathbf{f}) = +1$ but $Q(\mathbf{t}) = -1$, so no rotor maps one to the other. The best a boost can do is $\cosh\varphi\,\mathbf{f} + \sinh\varphi\,\mathbf{t}$, still of declared length 1, which keeps a formal part.
4. Estimate animal and food sense vectors from usage (for instance, contexts like *the lamb grazed* vs. *the lamb was tender*) for several words, and compare similarities within each sense. A shared rotor predicts equal similarities across the alternation. Systematically different similarities count against one shared program.
5. ⋆ The polar pair needs an axis with $\mathbf{e}^2=+1$, for the projectors; the seasons need a plane whose bivector squares to -1, which two $+1$ axes (or two -1 axes) provide. Swapped: on a -1 axis, $\mathbf{p}_+\mathbf{p}_- = \frac{1}{2}$, so alive and dead overlap; and a plane with one $+1$ and one -1 axis has $B^2=+1$, so SPRING would move along a hyperbola and never return.

Reading

- Clifford, *Applications of Grassmann's extensive algebra*, American Journal of Mathematics 1, 1878.
- Hestenes and Sobczyk, *Clifford Algebra to Geometric Calculus*, Reidel, 1984.
- Doran and Lasenby, *Geometric Algebra for Physicists*, Cambridge, 2003: chapters 2–4 for the product, reversion, and rotors.
- Dorst, Fontijne and Mann, *Geometric Algebra for Computer Science*, Morgan Kaufmann, 2007: the computational view of products and their evaluation.
- Abelson and Sussman, with Sussman, *Structure and Interpretation of Computer Programs*, MIT Press, 2nd ed. 1996: chapter 4, the evaluator as a program.
- Brehmer, de Haan, Behrends and Cohen, *Geometric Algebra Transformer*, NeurIPS 2023: the projective algebra $\mathrm{Cl}(3,0,1)$ in a neural network.
- Pustejovsky, *Functional Geometric Algebra* (position paper, 2026): the declared signature for meaning, Inner Application, and the boost analysis of good knife.